

ForeignKey Attribute in Entity Framework Core

In this article, I will discuss the **ForeignKey Data Annotation Attribute in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing the [Key Attributes in Entity Framework Core](#) with Examples. Before understanding the ForeignKey Attribute in EF Core, let us first understand what a foreign key is in a database.

What is a Foreign Key Constraint in a Database?

A Foreign Key Constraint in a database enforces a relationship between two tables. It is a field (or collection of fields) in one table that uniquely identifies a row of another table. The table that contains the foreign key is called the child table, and the table referenced by the foreign key is called the parent table.

To establish a relationship between two database tables, a foreign key must be specified in one table (known as the child table) that refers to a unique column (typically the Primary Key or Unique Key) in the other table (known as the parent table). Foreign keys ensure referential integrity constraints by preventing operations that would leave the database inconsistent. For example:

- It is not allowed to insert a row in the child table with a foreign key that doesn't exist in the parent table.
- Deleting a row in the parent table referenced by a child table would violate referential integrity unless cascading rules are defined.

Example: If Table A has a foreign key referencing Table B, every value in the foreign key column of Table A must exist as a primary key or unique key in Table B. This ensures no orphaned records (rows without a valid reference) exist.

What is a Dependent Entity and a Principal Entity in Entity Framework Core?

In Entity Framework Core, relationships between entities are defined in terms of Dependent Entities and Principal Entities:

- **Principal Entity:** The entity that contains the primary key (PK) or Unique Key in a relationship. It is considered the “parent” in the relationship.
- **Dependent Entity:** This is the entity that contains the foreign key (FK). Its identity and existence depend on the principal entity. It is considered the “child” entity in the relationship.

For example, if we have a relationship between Employee and Department, each employee can belong to a department, and each department can have multiple employees. Here,

- The Department is the Principal Entity because it holds the primary or Unique Key.
- The Employee is the Dependent Entity because it contains a foreign key pointing to the Department entity.

Examples to Understand Default Foreign Key Convention in EF Core:

The default foreign key convention in Entity Framework Core relies on the navigation properties defined in the entities. When navigation properties are present, EF Core can automatically determine the foreign key based on naming conventions. Let us understand this with an example. First, create the following Employee and Department Entities within the Entities folder.

Note: Before proceeding further, first of all, we need to identify the Principal Entity and the Dependent Entity. In our example, the Department is the Principal Entity, and the Employee is the Dependent Entity. In the Dependent Entity Employee, we will create the Foreign Key.

Employee.cs

Create a class file named **Employee.cs** within the **Entities** folder, and then copy and paste the following code. As you can see, the Employee class has a navigation property, Department, which links an employee to a department.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        // Navigation property
        public Department Department { get; set; }
        // Foreign key property
        public int DepartmentId { get; set; }
    }
}
```

Department.cs

Create a class file named **Department.cs** within the **Entities** folder and then copy and paste the following code. The Department class has a navigation property, Employees, representing the collection of employees in that department.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Department
    {
        public int Id { get; set; }
        public string Name { get; set; }
        // Navigation property
        public ICollection<Employee> Employees { get; set; }
    }
}
```

Default Foreign Key Convention in EF Core:

Entity Framework Core automatically recognizes the DepartmentId property in the Employee class as the foreign key for the relationship. EF Core follows a convention: The foreign key property name should match the navigation property name plus "Id." In this case:

- Navigation property: Department
- Foreign key property: DepartmentId

Since this naming convention is followed, there is no need to use the [ForeignKey] attribute or configure the foreign key with the Fluent API.

What EF Core Will Do:

EF Core will automatically configure a one-to-many relationship between Employee and Department:

- One Department can have many Employees.
- It will create a foreign key column, DepartmentId, in the Employee table, which will reference the Id column in the Department table.

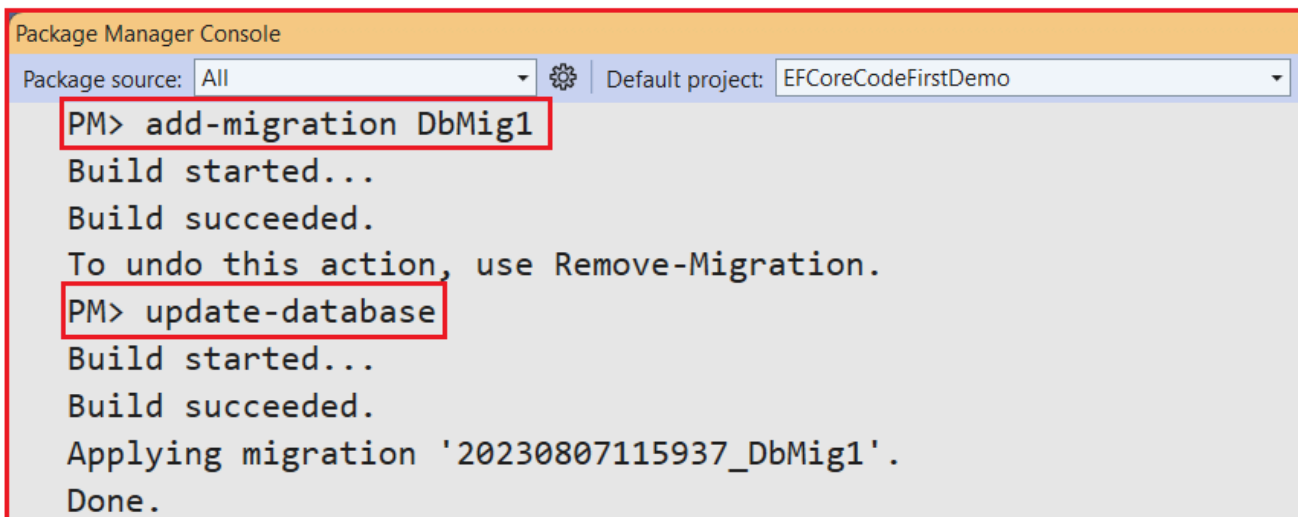
Modifying the Context Class:

Next, modify the **EFCoreDbContext** class as follows. Here, we will create a new database called EmployeeDB. We will create the Employees and Departments database tables within this database with one-to-many relationships between them, i.e., one department can have many employees.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Department> Departments { get; set; }
        public DbSet<Employee> Employees { get; set; }
    }
}
```

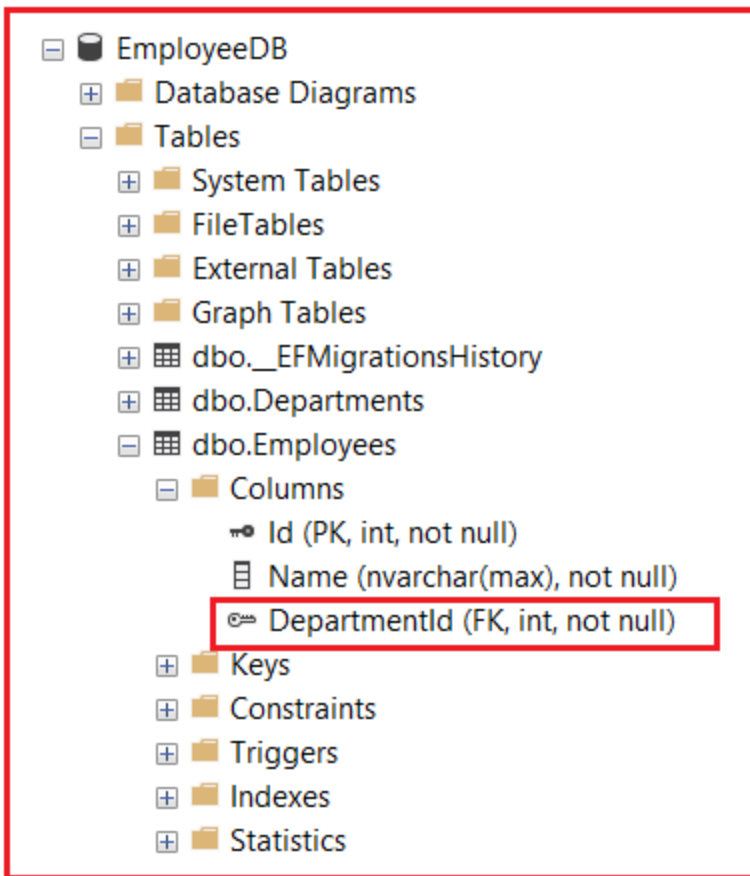
Generating Migration

So, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands as follows.

A screenshot of the Package Manager Console window in Visual Studio. The window has a title bar 'Package Manager Console' and a toolbar with 'Package source: All' and 'Default project: EFCoreCodeFirstDemo'. The console output shows the following commands and their results:

```
PM> add-migration DbMig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807115937_DbMig1'.
Done.
```

If you verify the database, the foreign key should have been created in the Employees database table, as shown in the image below. Here, you can see the Foreign Key Property is created with the name DepartmentId.



Creating Foreign Key without the Scaler Property:

Suppose the **Foreign Key Property**, i.e., the **Scaler Property**, which is **DepartmentId**, does not exist in the **Dependent Entity** class, i.e., in the **Employee** class. In that case, the Entity Framework Core will automatically create a Foreign Key column. Here, EF Core will:

- Create a foreign key column named **DepartmentId** in the **Employee** table because of the presence of the **Department** navigation property.
- It uses the naming convention **(Department + Id)** to create the foreign key column name.

To understand this better, please modify the **Employee Entity** class as follows. Here, you can see we have not added the **DepartmentId** property. So, in this case, Entity Framework Core will create the Foreign Key with the name **DepartmentId**. We should have added the **Department** navigation Property. Otherwise, the foreign key will not be created.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        //To Create a Foreign Key pointing to the Id column of the
        Departments table
        //it should have the Department Navigation Property
        public Department Department { get; set; }
        //In this case, EF Core automatically create the DepartmentId Foreign
        Key in the Employees table
    }
}
```

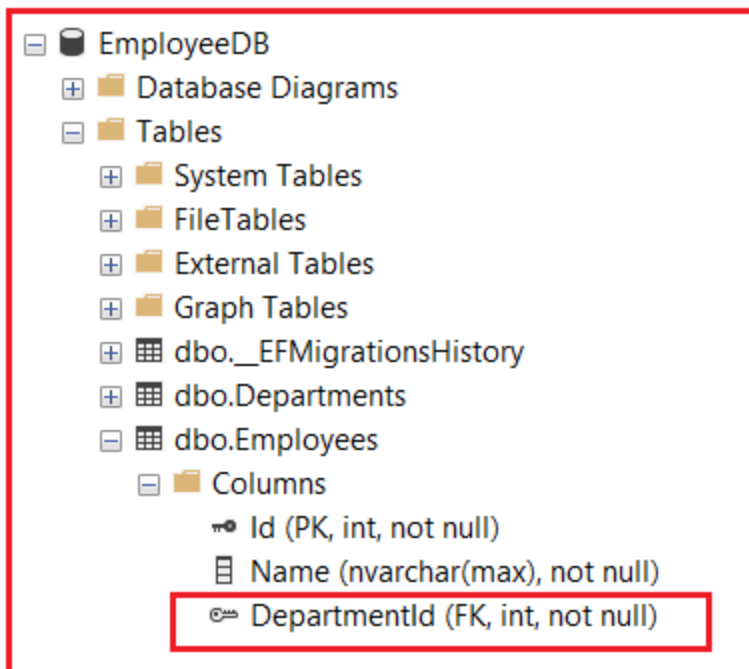
```
}  
}
```

With the above changes in place, open the Package Manager Console and Execute the following **add-migration** and **update-database** commands as shown below.



```
Package Manager Console  
Package source: All | Default project: EFCoreCodeFirstDemo  
PM> add-migration DbMig2  
Build started...  
Build succeeded.  
To undo this action, use Remove-Migration.  
PM> update-database  
Build started...  
Build succeeded.  
Applying migration '20230807133521_DbMig2'.  
Done.
```

If you verify the database, the foreign key should have been created in the Employees database table with the name DepartmentId, as shown in the image below.



These are the default conventions followed by Entity Framework Core to create the foreign key column in the database. Instead of using the default DepartmentId as the foreign key in the Employees table, we want to use a different column name for the foreign key. In that case, we need to override the default behavior using a Data Annotation Attribute or Fluent API Configuration. Let us proceed and understand how to configure the foreign key using the **ForeignKey Attribute** in Entity Framework Core.

What is ForeignKey Attribute in Entity Framework?

The [ForeignKey] attribute in Entity Framework Core specifies which property in the dependent entity should be treated as the foreign key for a relationship. This attribute is useful when the default conventions do not meet the business

requirements or when we want to clarify the relationship explicitly. If you go to the definition of the ForeignKey Attribute, you will see the following signature.

```
namespace System.ComponentModel.DataAnnotations.Schema
{
    ... public class ForeignKeyAttribute : Attribute
    {
        ... public ForeignKeyAttribute(string name);
        ... public string Name { get; }
    }
}
```

The **[ForeignKey]** attribute takes a string parameter that represents the name of the Reference Navigation Property or the scalar Foreign Key Property, depending on where the attribute is placed. So, there are two main ways to apply the [ForeignKey] attribute:

- On the **Foreign Key Scalar Property** in the dependent entity.
- On the **Reference Navigation Property** in the dependent entity.

Let's look at each use case in detail:

[ForeignKey] on the Foreign Key Scalar Property in the Dependent Entity

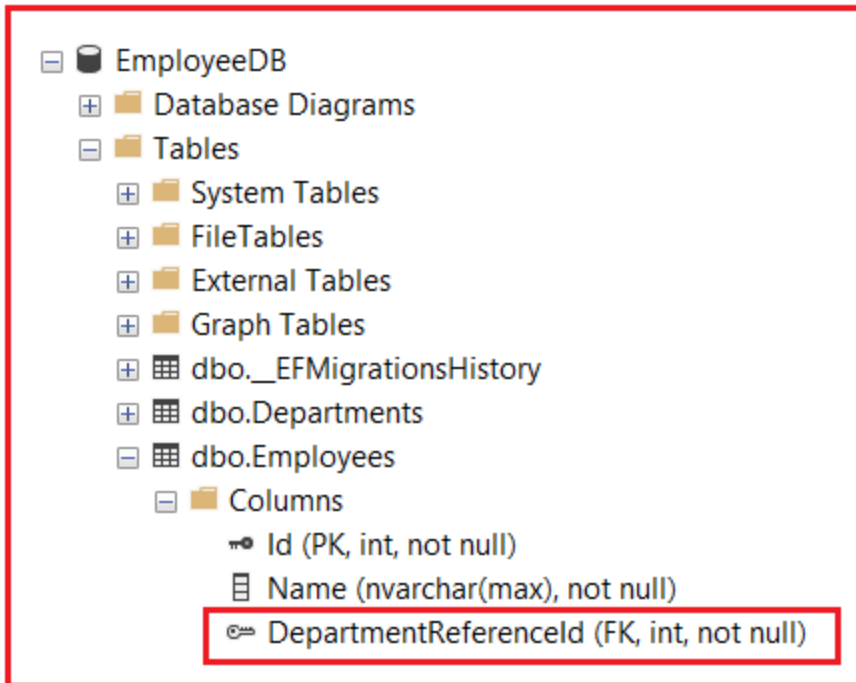
In this case, the [ForeignKey] attribute is applied to the foreign key scalar property of the dependent entity (the entity that holds the foreign key). In this case, we need to specify the related reference navigation property name in the [ForeignKey] attribute. For a better understanding, please modify the Employee Entity as follows.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        [ForeignKey("Department")]
        public int DepartmentReferenceId { get; set; }
        //Related Standard Navigational Property
        public Department Department { get; set; }
    }
}
```

In the above code, the **[ForeignKey("Department")]** attribute is applied to the **DepartmentReferenceId** scalar property. This tells EF Core that DepartmentReferenceId is the foreign key linking the Employee entity to the Department entity. EF Core will create a foreign key column, **DepartmentReferenceId**, in the **Employees** table pointing to the **Id** column of the **Departments** table.

With the above changes in place, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. Once you execute the command, verify the database. The foreign key should have been created

in the **DepartmentReferenceId** column of the **Employees** table, as shown in the image below.



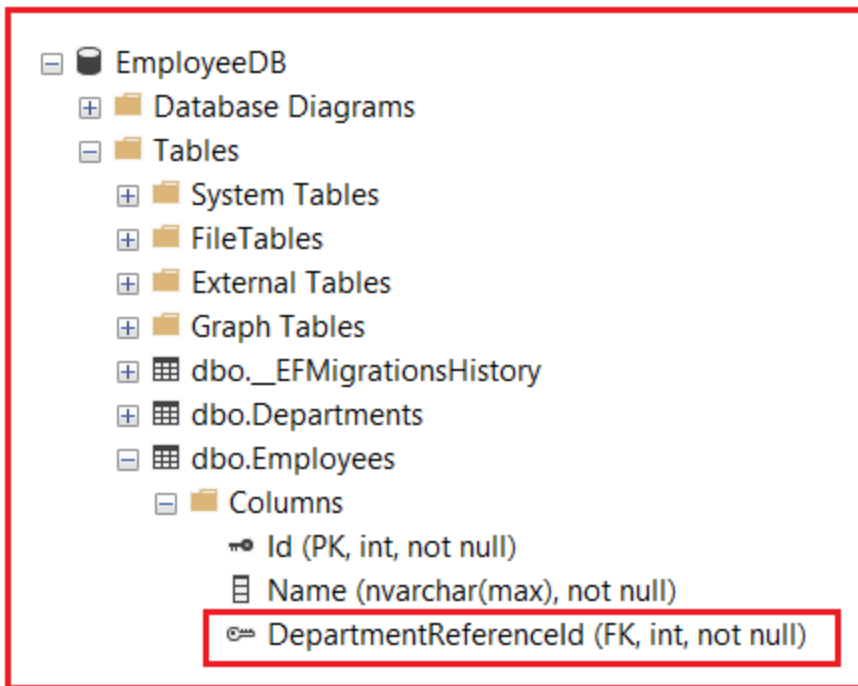
[ForeignKey] on the Navigation Property in the Dependent Entity

We can also apply the [ForeignKey] attribute to the reference navigation property in the dependent entity. In this case, to the [ForeignKey] attribute constructor, we need to specify the name of the foreign key scalar property of the dependent entity. For a better understanding, please modify the Employee Entity as follows.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int DepartmentReferenceId { get; set; }
        [ForeignKey("DepartmentReferenceId")]
        public Department Department { get; set; }
    }
}
```

Here, the **[ForeignKey("DepartmentReferenceId")]** attribute is applied to the **Department** reference navigation property. This tells EF Core to serve the **DepartmentReferenceId** as the foreign key linking the **Employee** to the **Department**.

With the above changes in place, open the Package Manager Console and Execute the **Add-Migration** and **Update-Database** commands. Once the commands are executed, verify the database. The foreign key should have been created in the **Employees** table with the name **DepartmentReferenceId**, as shown in the image below.



Composite Foreign Key using EF Core:

A Composite Foreign Key is a foreign key in a dependent table that consists of multiple columns referencing a composite primary key in a principal table. A composite foreign key is used to maintain referential integrity constraints between two tables when the primary key of the principal table is made up of more than one column (i.e., a composite primary key).

Real-time Scenario:

Let's see a real-time example to understand the Composite Foreign Keys in the Entity Framework Core, where the Principal Entity has a Composite Primary Key, and the Dependent Entity points to it using composite foreign keys. Imagine we have a scenario in an Order Management System where:

- **Order** is a **Principal Entity** with a composite primary key (**OrderId**, **CustomerId**).
- **OrderItem** is a **Dependent Entity** with a composite foreign key pointing to **OrderId** and **CustomerId** in the Order entity.

Let us proceed and see how we can implement this.

Order Entity (Principal Entity with Composite Primary Key)

Create a class file named **Order.cs** within the Entities folder and then copy and paste the following code. In the following entity, OrderId and CustomerId together form the composite primary key for the Order entity. The **[PrimaryKey]** attribute marks both properties as part of the primary key.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [PrimaryKey("OrderId", "CustomerId")] //Composite Primary Key
    public class Order
    {
        public int OrderId { get; set; }
```

```

        public int CustomerId { get; set; }
        public DateTime OrderDate { get; set; }
        // Navigation property for the dependent entity
        public ICollection<OrderItem> OrderItems { get; set; }
    }
}

```

OrderItem Entity (Dependent Entity with Composite Foreign Key)

Create a class file named **OrderItem.cs** within the Entities folder, and then copy and paste the following code. To define a composite foreign key using Data Annotations, we need to apply the **[ForeignKey]** attribute on the Reference Navigation property and provide a comma-separated list of the foreign key properties.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int OrderItemId { get; set; }
        // Foreign key properties
        public int OrderId { get; set; }
        public int CustomerId { get; set; }
        public string ProductName { get; set; }
        public int Quantity { get; set; }
        // Apply ForeignKey attribute on the navigation property, pointing to
multiple foreign key properties
        [ForeignKey(nameof(OrderId) + "," + nameof(CustomerId))]
        public Order Order { get; set; }
    }
}

```

Explanation:

- The [ForeignKey] attribute is applied once on the navigation property (Order).
- The value passed to the [ForeignKey] attribute is a comma-separated list of foreign key properties (OrderId, CustomerId), which form the composite foreign key together.

DbContext Class:

Next, modify the EFCoreDbContext class as follows. Here, we are creating a new database, OrderDB, and within this database, we are creating the Orders and OrderItems tables.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {

```

```

        optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServer
    }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
}
}

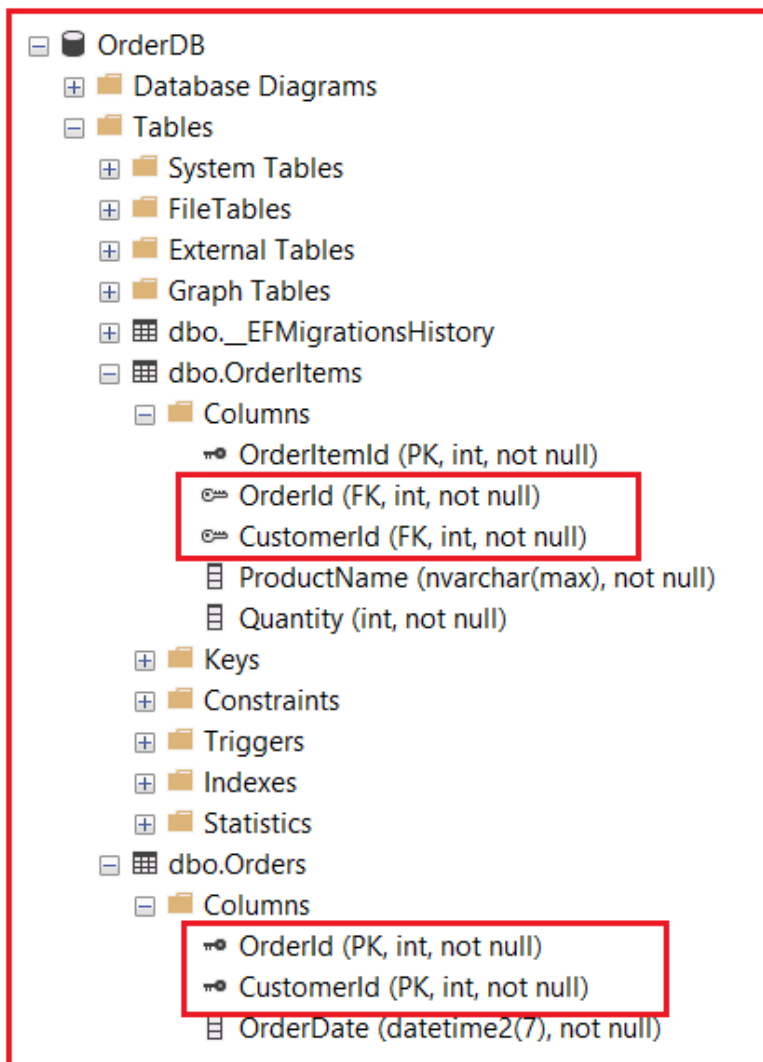
```

Migration and Database Creation

Run the following commands in the Package Manager Console to generate the migration and create the database:

- **Add-Migration CompositeKeyExample**
- **Update-Database**

This will create the **Orders** and **OrderItems** tables in the **OrderDB** database, where the Orders table will have a composite primary key (OrderId, CustomerId), and the OrderItems table will have composite foreign keys (OrderId, CustomerId) pointing to the Orders table as shown in the below image:



Inserting Data into Composite Primary Key and Foreign Keys:

Let us see how to insert data into the above two tables. For this, please modify the Program class as follows:

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            try
            {
                using (var context = new EFCoreDbContext())
                {
                    // Creating a new order
                    var order = new Order
                    {
                        OrderId = 1,
                        CustomerId = 101,
                        OrderDate = DateTime.Now
                    };
                    Console.WriteLine($"Creating Order: OrderId =
{order.OrderId}, CustomerId = {order.CustomerId}, OrderDate =
{order.OrderDate}");
                    // Adding the order to the context
                    context.Orders.Add(order);
                    Console.WriteLine("Order has been added to the
context.");
                    // Creating the first order item
                    var orderItem1 = new OrderItem
                    {
                        OrderId = 1,           // Composite foreign key part 1
                        CustomerId = 101,       // Composite foreign key part 2
                        ProductName = "Laptop",
                        Quantity = 2
                    };
                    Console.WriteLine($"Creating OrderItem 1: ProductName =
{orderItem1.ProductName}, Quantity = {orderItem1.Quantity}");
                    // Adding the first order item to the context
                    context.OrderItems.Add(orderItem1);
                    Console.WriteLine("OrderItem 1 has been added to the
context.");
                    // Creating the second order item
                    var orderItem2 = new OrderItem
                    {
                        OrderId = 1,           // Composite foreign key part 1
                        CustomerId = 101,       // Composite foreign key part 2
                        ProductName = "Desktop",
                        Quantity = 1
                    };
                    Console.WriteLine($"Creating OrderItem 2: ProductName =
{orderItem2.ProductName}, Quantity = {orderItem2.Quantity}");
```

```

        // Adding the second order item to the context
        context.OrderItems.Add(orderItem2);
        Console.WriteLine("OrderItem 2 has been added to the
context.");

        // Saving all changes to the database
        context.SaveChanges();
        Console.WriteLine("Changes have been saved to the
database.");
    }
    Console.WriteLine("Order Placed Successfully.");
}
catch (Exception ex)
{
    // Handling any exceptions that occur during the execution
    Console.WriteLine($"An error occurred: {ex.Message}");
}
}
}
}
}

```

Output:

```

Creating Order: OrderId = 1, CustomerId = 101, OrderDate = 11-09-2024 11:45:34
Order has been added to the context.
Creating OrderItem 1: ProductName = Laptop, Quantity = 2
OrderItem 1 has been added to the context.
Creating OrderItem 2: ProductName = Desktop, Quantity = 1
OrderItem 2 has been added to the context.
Changes have been saved to the database.
Order Placed Successfully.

```

Note: In our upcoming articles, I will discuss One-to-One, One-to-Many, and Many-to-Many Relationships using Entity Framework Core.

When Should We Use ForeignKey Attribute in Entity Framework Core?

We need to use the [ForeignKey] attribute in the following scenarios:

- **Non-Standard Naming:** When your foreign key property doesn't follow the default naming conventions (e.g., using DepartmentReferenceId instead of DepartmentId).
- **Explicit Relationship Definition:** To clearly specify which property serves as the foreign key, enhancing code readability and maintainability.
- **Composite Keys:** When defining relationships that involve composite keys (multiple foreign key properties).
- **Multiple Relationships Between Entities:** When an entity has multiple navigation properties pointing to the same principal entity, and you need to specify distinct foreign keys for each relationship.

In the next article, I will discuss [Index Attributes in Entity Framework Core](#) with Examples. In this article, I try to explain the **ForeignKey Data Annotation Attribute in Entity Framework Core** with examples. I hope you enjoyed this ForeignKey Attribute in EF Core with Examples article. Please provide your valuable feedback and suggestions for this article.